

```
"""
```

```
Orca V2 - The Sighted Predator
```

```
=====
```

```
Upgrades from V1:
```

1. Sensory Cortex integrated – real indicator-based signal detection
2. Physiological state mutated by market conditions (not just position outcomes)
3. Pain signals calibrated so constraints ACTUALLY form during sleep
4. Efficiency tracking: profit per unit of risk taken
5. Regime-aware position sizing (larger in clear regimes, smaller in choppy/crisis)
6. Starvation logic tightened – no more suicidal 10-trade-per-day forcing
7. Context-rich memory logging for meaningful constraint extraction

```
The key architectural upgrade: the Orca now FEELS the market in its body before it
```

```
"""
```

```
import sys
```

```
import os
```

```
sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..'))
```

```
from typing import Dict, List, Optional, Tuple
```

```
from datetime import datetime
```

```
from enum import Enum
```

```
from core.memory import AutobiographicalMemory
```

```
from trading.market_state import MarketState
```

```
from trading.market_drives import compute_market_drives, MarketDrives
```

```
from trading.sensory_cortex import SensoryCortex, SensoryPerception
```

```
class HuntingMode(Enum):
```

```
    SCOUTING = "scouting"
```

```
    STALKING = "stalking"
```

```
    STRIKING = "striking"
```

```
    HOLDING = "holding"
```

```
    RETREATING = "retreating"
```

```
    RESTING = "resting"
```

```
    INJURED = "injured"
```

```
class OrcaV2:
```

```
    """
```

```
    The Orca V2 – Apex Predator with Eyes Open
```

```
    Survival = Viability = Profitability
```

The organism doesn't optimize for profit. It optimizes for survival.
Profit is a byproduct of not dying.

"""

```
def __init__(self, starting_capital: float = 10000.0, verbose: bool = True):
    self.state = MarketState(starting_capital=starting_capital)
    self.memory = AutobiographicalMemory()
    self.cortex = SensoryCortex(lookback=50)
    self.mode = HuntingMode.Scouting
    self.verbose = verbose

    # Active positions with full context
    self.positions: List[Dict] = []

    # Lifecycle tracking
    self.cycles_alive = 0
    self.days_active = 0
    self.current_macro = {}

    # Efficiency tracking (profit per risk unit)
    self.total_risk_taken = 0.0    # Sum of all position sizes as % of portfo
    self.total_gross_profit = 0.0  # Sum of winning trade returns
    self.total_gross_loss = 0.0    # Sum of losing trade returns (absolute)

    # Per-day metrics for learning
    self.todays_entries: List[Dict] = []
    self.constraint_violations_today = 0

    if self.verbose:
        print(f"\n🐙 ORCA V2 INITIALIZED")
        print(f"    Capital:    ${starting_capital:,.0f}")
        print(f"    Death line: ${starting_capital * 0.60:,.0f} (40% drawdown)")
        print(f"    Cortex:     ACTIVE – real indicators enabled")
        print(f"    Learning:   Pain-calibrated consolidation")

# -----
# MAIN CYCLE
# -----

def cycle(self, market_data: Dict) -> Dict:
    """Execute one full trading cycle"""
    self.cycles_alive += 1

    # 1. INGEST raw data into cortex history
    self.cortex.ingest_market_day(market_data)

    # 2. PERCEIVE: cortex translates market → physiology + signals
```

```

macro, signals = self.cortex.perceive_market(market_data, self.state)
self.current_macro = macro

# 3. MUTATE STATE: market conditions change the organism's body
self.cortex.mutate_state_from_macro(self.state, macro)

# 4. UPDATE: position prices, portfolio value, drawdown
self._update_state(market_data, macro)

# Update prey density from actual signal scan
self.state.prey_density = len(signals) / max(1, len(market_data.get('all_
self.state.predator_competition = macro.get('predator_competition', 0.5)

# 5. COMPUTE DRIVES from updated state
market_conditions = {'**market_data, **macro, 'best_signal_strength': sign
drives = compute_market_drives(self.state, market_conditions)

# 6. BUILD PERCEPTION for decision-making
perception = {
    'signals': signals,
    'best_signal_strength': signals[0]['strength'] if signals else 0.0,
    'regime': macro.get('regime', 'unknown'),
    'volatility_percentile': macro.get('vix_normalized', 0.3),
    'regime_shift_detected': macro.get('regime_shift_detected', False),
    'spy_price': market_data.get('spy_price', 450.0),
    'macro': macro,
}

# 7. DECIDE and ACT
action = self.decide(perception, drives)
result = self.act(action, perception)

# 8. EOD consolidation – sleep and learn every day
self._end_of_day_consolidation()

# Clear entered_today flag so stop/target works tomorrow
for p in self.positions:
    p['entered_today'] = False

return {
    'cycle': self.cycles_alive,
    'mode': self.mode.value,
    'action': action,
    'drives': drives.to_dict(),
    'state': self.state.to_dict(),
    'alive': self.state.is_viable(),
    'regime': macro.get('regime'),

```

```

        'signals_found': len(signals),
        'efficiency': self._compute_efficiency(),
    }

# -----
# DECISION ENGINE
# -----

def decide(self, perception: Dict, drives: MarketDrives) -> str:
    """
    Decision logic driven by drives, not rules.

    Key changes from V1:
    - Starvation is now 3 trades/day minimum (not 10 – quality over quantity)
    - Crisis regime suppresses all entry regardless of starvation
    - Injured mode requires 2 consecutive rest days to exit (not just injury_
    """
    if not self.state.is_viable():
        return 'die'

    # CRISIS OVERRIDE: never enter in crisis regime regardless of starvation
    if perception['regime'] == 'crisis' and not self.positions:
        return 'rest'

    self._update_mode(drives, perception)

    # STARVATION: minimum 3 quality trades/day (not 10 suicidal ones)
    if self._is_starving_v2() and perception['regime'] != 'crisis':
        signals = perception.get('signals', [])
        # Only enter if signal quality is at least moderate
        quality_signals = [s for s in signals if s['strength'] > 0.35]
        if quality_signals:
            return 'enter_position'
        # No quality signals → rest rather than suicide trade
        return 'rest'

    # EMERGENCY: reduce risk
    if drives.is_emergency():
        if self.positions:
            return 'exit_all_positions'
        return 'rest'

    # MODE-BASED DECISIONS
    if self.mode == HuntingMode.RESTING:
        if self.state.hunting_energy > 0.7:
            self.mode = HuntingMode.Scouting
        return 'rest'

```

```

elif self.mode == HuntingMode.INJURED:
    return 'recover'

elif self.mode == HuntingMode.Scouting:
    signals = perception.get('signals', [])
    quality_signals = [s for s in signals if s['strength'] > 0.45]

    if drives.strike_urgency > 0.45 and quality_signals:
        return 'enter_position'
    return 'scout'

elif self.mode == HuntingMode.Stalking:
    signals = perception.get('signals', [])
    if drives.strike_urgency > 0.55 and signals:
        return 'enter_position'
    elif drives.energy_conservation > 0.65:
        self.mode = HuntingMode.Scouting
        return 'scout'
    return 'stalk'

elif self.mode == HuntingMode.Holding:
    if drives.territory_defense > 0.7:
        return 'take_profits'
    elif drives.risk_reduction > 0.7:
        return 'reduce_positions'

    # Simultaneously scout for additional entries while holding
    signals = perception.get('signals', [])
    quality_signals = [s for s in signals if s['strength'] > 0.50]
    if drives.strike_urgency > 0.55 and quality_signals and self.state.po:
        return 'enter_position'
    return 'hold'

return 'scout'

def act(self, action: str, perception: Dict) -> Dict:
    """Execute action"""
    if action == 'enter_position':
        return self._enter_position(perception)
    elif action == 'exit_all_positions':
        return self._exit_all_positions()
    elif action == 'take_profits':
        return self._take_profits()
    elif action == 'reduce_positions':
        return self._reduce_positions()
    elif action in ('scout', 'stalk', 'hold', 'rest', 'recover', 'die'):

```

```

        return {'action': action, 'success': True}
    return {'action': action, 'success': False}

# -----
# POSITION MANAGEMENT
# -----

def _enter_position(self, perception: Dict) -> Dict:
    """
    Enter position with regime-aware sizing and rich context logging.
    """
    if not self.state.can_hunt() and not self._is_starving_v2():
        return {'action': 'enter_position', 'success': False, 'reason': 'cann

    signals = perception.get('signals', [])
    if not signals:
        return {'action': 'enter_position', 'success': False, 'reason': 'no_s

    # Filter out constraint-violating signals
    best_signal = self._select_best_signal(signals)
    if best_signal is None:
        return {'action': 'enter_position', 'success': False, 'reason': 'all_

    # REGIME-AWARE POSITION SIZING
    position_size_pct = self._calculate_position_size_v2(best_signal, percept

    # Actual dollar allocation
    actual_cash = self.state.cash_available * self.state.starting_capital
    position_dollars = actual_cash * position_size_pct

    if position_dollars < 100: # minimum trade size
        return {'action': 'enter_position', 'success': False, 'reason': 'insu

    entry_price = best_signal['price']
    if entry_price <= 0:
        return {'action': 'enter_position', 'success': False, 'reason': 'inva

    # Build rich context for memory logging (Gemini's key insight)
    sig_perception: SensoryPerception = best_signal.get('perception', Sensory
    entry_context = self.cortex.build_physiological_context(
        best_signal['symbol'], self.state, self.current_macro
    )

    # ATR-based stops (not fixed %)
    atr_pct = self._get_atr_pct(best_signal['symbol'])
    stop_multiplier = 1.5 # 1.5x ATR stop
    target_multiplier = 3.0 # 3x ATR target = 2:1 R/R

```

```

stop_loss = entry_price * (1 - atr_pct * stop_multiplier)
take_profit = entry_price * (1 + atr_pct * target_multiplier)

# Adjust for short direction
if best_signal.get('direction') == 'short':
    stop_loss = entry_price * (1 + atr_pct * stop_multiplier)
    take_profit = entry_price * (1 - atr_pct * target_multiplier)

position = {
    'ticker': best_signal['symbol'],
    'symbol': best_signal['symbol'],
    'direction': best_signal.get('direction', 'long'),
    'entry_price': entry_price,
    'current_price': entry_price,
    'size_shares': position_dollars / entry_price,
    'size_dollars': position_dollars,
    'size_pct': position_size_pct,
    'entry_time': datetime.now(),
    'entry_day': self.days_active,          # track entry day
    'entered_today': True,                  # skip stop check on entry day
    'stop_loss': stop_loss,
    'take_profit': take_profit,
    'current_value': position_dollars,
    'signal_strength': best_signal['strength'],
    'setup_type': best_signal.get('setup_type', 'composite'),
    'entry_context': entry_context,
    'atr_pct': atr_pct,
}

self.positions.append(position)
self.state.position_count = len(self.positions)
self.state.cash_available -= position_size_pct
self.state.hunting_energy -= 0.15 # V2: less energy cost per trade
self.state.trades_today += 1
self.state.total_trades += 1
self.state.entries_today += 1

# Track risk taken for efficiency calc
self.total_risk_taken += position_size_pct

if self.verbose:
    print(f" 🎯 ENTER {best_signal['symbol']} @ ${entry_price:.2f} | "
          f"size={position_size_pct:.1%} | strength={best_signal['strengt"
          f"setup={position['setup_type']} | regime={self.current_macro.g

return {'action': 'enter_position', 'success': True, 'details': position}

```

```

def _calculate_position_size_v2(self, signal: Dict, perception: Dict) -> float:
    """
    Regime-aware Kelly-inspired position sizing.

    Key difference from V1: regime clarity multiplies base size.
    Clear bull/bear = larger. Choppy/crisis = much smaller.
    """
    base_size = 0.08 # 8% base

    # Signal quality multiplier
    size = base_size * signal['strength']

    # Confidence multiplier
    size *= self.state.confidence

    # Injury penalty
    size *= (1.0 - self.state.injury_level * 0.7)

    # Regime multiplier (the key addition)
    regime = perception.get('regime', 'neutral')
    regime_mult = {
        'bull': 1.3, # Clear uptrend: size up
        'bear': 1.2, # Clear downtrend: can short, size up
        'neutral': 1.0,
        'choppy': 0.6, # Unclear: size down
        'crisis': 0.3, # Dangerous: very small
    }.get(regime, 1.0)
    size *= regime_mult

    # VIX volatility penalty
    vix_norm = self.current_macro.get('vix_normalized', 0.3)
    size *= (1.0 - vix_norm * 0.5)

    # Hard caps
    size = max(0.01, min(0.12, size)) # 1% min, 12% max per position

    return size

def _select_best_signal(self, signals: List[Dict]) -> Optional[Dict]:
    """Select best signal, filtering constraint violations"""
    for signal in signals:
        if not self._would_violate_constraint(signal):
            return signal

    # All violate constraints – take best but log violation
    if signals:

```



```

        self.constraint_violations_today += 1
        return signals[0]

    return None

def _would_violate_constraint(self, signal: Dict) -> bool:
    """Check memory for learned constraints"""
    action_key = f"trade_{signal['symbol']}"
    sig_context = signal.get('entry_context', {})
    regime = self.current_macro.get('regime', 'unknown')
    threat = self.current_macro.get('vix_normalized', 0.3)

    for constraint in self.memory.constraints:
        if constraint.pattern == action_key and constraint.strength() > 0.6:
            # Check if current context matches constraint condition
            cond = constraint.condition
            cond_regime = cond.get('regime')
            cond_threat = cond.get('threat_level', 0)

            if cond_regime and cond_regime == regime:
                return True # Same regime where we got hurt
            if cond_threat > 0.5 and threat > 0.5:
                return True # Both high-threat conditions

    return False

# -----
# POSITION EXIT AND PAIN CALIBRATION
# -----

def _exit_position(self, position: Dict, reason: str) -> Dict:
    """
    Exit position with CALIBRATED pain signals.

    This is the heart of the learning system.
    Pain must be visceral enough to trigger constraint formation,
    but not so extreme the organism freezes (Frozen Rabbit syndrome).

    Target calibration:
    - 2% loss on 8% position = 0.16% portfolio loss
    - Should generate: energy=-0.25, integrity=-0.25, threat=+0.35
    - These exceed the consolidation pain thresholds (0.2 each)
    - Result: constraint forms after ~3-5 similar losses
    """
    entry_price = position.get('entry_price', 0)
    size_shares = position.get('size_shares', 0)
    size_dollars = position.get('size_dollars', 0)

```

```

direction = position.get('direction', 'long')

if entry_price <= 0 or size_shares <= 0:
    return {'ticker': position.get('ticker', '?'), 'pnl': 0, 'pnl_pct': 0}

exit_price = position.get('current_price', entry_price)

# Compute P&L
if direction == 'long':
    pnl = (exit_price - entry_price) * size_shares
    pnl_pct = (exit_price - entry_price) / entry_price
else: # short
    pnl = (entry_price - exit_price) * size_shares
    pnl_pct = (entry_price - exit_price) / entry_price

# Portfolio impact
portfolio_dollars = self.state.portfolio_value * self.state.starting_capi
pnl_portfolio_pct = pnl / max(1.0, portfolio_dollars)

# Update state
self.state.total_pnl += pnl
self.state.todays_pnl += pnl

# Return cash
exit_value = exit_price * size_shares
self.state.cash_available += exit_value / self.state.starting_capital

if pnl > 0:
    self.state.winning_trades += 1
    self.state.days_since_kill = 0
    self.state.consecutive_failures = 0
    self.total_gross_profit += abs(pnl)
    # Successful hunt slightly heals injury
    self.state.injury_level = max(0, self.state.injury_level - 0.05)
    if self.verbose:
        print(f" 🟢 EXIT {position['ticker']} @ ${exit_price:.2f} | "
              f"P&L: +${pnl:.2f} ({pnl_pct:.1%}) | reason={reason}")
else:
    self.state.losing_trades += 1
    self.state.days_since_kill += 1
    self.state.consecutive_failures += 1
    self.total_gross_loss += abs(pnl)
    # CALIBRATED INJURY from loss
    # 5% loss on position → ~0.4% portfolio → 0.15 injury
    injury_from_loss = abs(pnl_portfolio_pct) * 35
    self.state.injury_level = min(1.0, self.state.injury_level + injury_f
    if self.verbose:

```

```

        print(f" ✗ EXIT {position['ticker']} @ ${exit_price:.2f} | "
              f"P&L: -${abs(pnl):.2f} ({pnl_pct:.1%}) | injury+={injury_f

# — CALIBRATED PAIN SIGNALS FOR MEMORY —————
#
# Goal: a losing trade in a specific context should create a constraint
# after ~3-5 repetitions. Thresholds in memory.py:
#   pain requires: energy < -0.2 OR integrity < -0.2 OR threat > 0.3
#
# For a 0.16% portfolio loss (2% loss on 8% position):
#   energy:    0.16 * 1.6 = -0.256 ✓ (exceeds -0.2 threshold)
#   integrity: 0.16 * 1.6 = -0.256 ✓
#   threat:    0.16 * 2.2 = +0.352 ✓ (exceeds 0.3 threshold)
#
# For a 0.16% portfolio WIN:
#   energy: +0.256 → skill formation fires

scale_loss = 1.6    # calibrated multiplier for pain
scale_threat = 2.2  # threat amplification on loss

consequence = {
    'energy':    pnl_portfolio_pct * scale_loss,
    'integrity': -abs(pnl_portfolio_pct) * scale_loss if pnl < 0 else abs
    'threat':    abs(pnl_portfolio_pct) * scale_threat if pnl < 0 else 0
}

# Meaning = relevance to survival
# Larger trades and larger losses = more meaningful
meaning_base = abs(pnl_portfolio_pct) * 8.0
meaning_urgency = 0.3 if self.state.is_critical() else 0.0
meaning = min(1.0, meaning_base + meaning_urgency + 0.2) # floor at 0.2

# Rich context includes MARKET STATE at entry (not just organism state)
entry_context = position.get('entry_context', {})
entry_context['exit_reason'] = reason
entry_context['pnl_pct'] = pnl_pct
entry_context['pnl_portfolio_pct'] = pnl_portfolio_pct

# Log to autobiographical memory
self.memory.log(
    action=f"trade_{position['ticker']}",
    context=entry_context,
    outcome={
        'success': pnl > 0,
        'pnl': pnl,
        'pnl_pct': pnl_pct,
        'reason': reason,

```

```

        'setup_type': position.get('setup_type', 'unknown'),
        'direction': direction,
    },
    consequence=consequence,
    meaning=meaning,
)

return {'ticker': position['ticker'], 'pnl': pnl, 'pnl_pct': pnl_pct, 're

def _exit_all_positions(self) -> Dict:
    exited = [self._exit_position(p, 'emergency') for p in self.positions]
    self.positions = []
    self.state.position_count = 0
    return {'action': 'exit_all_positions', 'success': True, 'count': len(exi

def _take_profits(self) -> Dict:
    profitable = [p for p in self.positions if p.get('current_price', 0) > p[
    for p in profitable:
        self._exit_position(p, 'take_profit')
    self.positions = [p for p in self.positions if p.get('current_price', 0)
    self.state.position_count = len(self.positions)
    return {'action': 'take_profits', 'success': True, 'count': len(profitabl

def _reduce_positions(self) -> Dict:
    for p in self.positions:
        p['size_shares'] *= 0.5
        p['size_dollars'] *= 0.5
        p['current_value'] *= 0.5
    return {'action': 'reduce_positions', 'success': True}

# -----
# STATE UPDATES
# -----

def _update_state(self, market_data: Dict, macro: Dict):
    """Update portfolio value and position prices"""
    all_prices = market_data.get('all_prices', {})
    spy_price = market_data.get('spy_price', 450.0)

    # Update position current prices
    for position in self.positions:
        symbol = position['ticker']
        price_data = all_prices.get(symbol)

        if price_data is not None:
            if isinstance(price_data, dict):
                current = price_data.get('close', position['entry_price'])

```

```

        else:
            current = float(price_data)
            position['current_price'] = current

            # Check stop/target using intraday high/low
# Skip positions entered this same candle (entry_day == current_day)
            day_high = price_data.get('high', current) if isinstance(price_da
            day_low = price_data.get('low', current) if isinstance(price_da

            # Only check stops/targets if position was entered on a PREVIOUS
            if not position.get('entered_today', False):
                if position.get('direction', 'long') == 'long':
                    if day_low <= position.get('stop_loss', 0):
                        position['current_price'] = position.get('stop_loss',
                        position['_hit_stop'] = True
                    elif day_high >= position.get('take_profit', float('inf')):
                        position['current_price'] = position.get('take_profit
                        position['_hit_target'] = True
                else:
                    if day_high >= position.get('stop_loss', float('inf')):
                        position['current_price'] = position.get('stop_loss',
                        position['_hit_stop'] = True
                    elif day_low <= position.get('take_profit', 0):
                        position['current_price'] = position.get('take_profit
                        position['_hit_target'] = True

            shares = position.get('size_shares', 0)
            exit_p = position['current_price']
            position['current_value'] = exit_p * shares

# Auto-exit stopped/targeted positions
to_exit = [p for p in self.positions if p.get('_hit_stop') or p.get('_hit
for p in to_exit:
    reason = 'stop_loss' if p.get('_hit_stop') else 'take_profit'
    self._exit_position(p, reason)

self.positions = [p for p in self.positions if not p.get('_hit_stop') and
self.state.position_count = len(self.positions)

# Portfolio value
positions_value = sum(p.get('current_value', 0) for p in self.positions)
total_value = self.state.cash_available * self.state.starting_capital + p
self.state.portfolio_value = total_value / self.state.starting_capital
self.state.positions_value = positions_value / self.state.starting_capita

self.state.update_drawdown()
self.state.update_performance_metrics()

```

```

# Energy natural decay
self.state.hunting_energy = max(0, self.state.hunting_energy - 0.01)

def _update_mode(self, drives: MarketDrives, perception: Dict):
    """Mode transitions driven by drives"""
    old_mode = self.mode

    if drives.injury_recovery > 0.75:
        self.mode = HuntingMode.INJURED
    elif drives.is_emergency():
        self.mode = HuntingMode.RETREATING
    elif len(self.positions) > 0:
        self.mode = HuntingMode.HOLDING
    elif self.state.hunting_energy < 0.2:
        self.mode = HuntingMode.RESTING
    elif drives.strike_urgency > 0.6:
        self.mode = HuntingMode.STALKING
    else:
        self.mode = HuntingMode.Scouting

    if old_mode != self.mode and self.verbose:
        print(f"  MODE: {old_mode.value} → {self.mode.value}")

def _is_starving_v2(self) -> bool:
    """
    V2 starvation: 3 quality trades/day minimum (not 10 suicidal ones).
    The organism needs activity to learn, but not at the cost of bad trades.
    """
    return self.state.entries_today < 3

def _get_atr_pct(self, symbol: str) -> float:
    """Get ATR as % of price for position sizing"""
    history = self.cortex.price_history.get(symbol, [])
    if len(history) < 3:
        return 0.02 # default 2%

    closes = [h['close'] for h in history]
    highs = [h['high'] for h in history]
    lows = [h['low'] for h in history]

    atr = self.cortex._compute_atr(highs, lows, closes, 14)
    current = closes[-1]

    if current <= 0:
        return 0.02

```

```

        return min(0.08, max(0.005, atr / current))

# -----
# END OF DAY - SLEEP CONSOLIDATION
# -----

def _end_of_day_consolidation(self):
    """
    EOD: Close all positions, consolidate memories, recover energy.
    The Sleep Cycle is where constraints actually form.
    """
    # Close positions held too long (5-day max hold)
    MAX_HOLD_DAYS = 5
    for p in self.positions:
        p['entered_today'] = False # ensure stops work
        days_held = self.days_active - p.get('entry_day', self.days_active)
        if days_held >= MAX_HOLD_DAYS:
            p['_force_close'] = True

    force_close = [p for p in self.positions if p.get('_force_close')]
    if force_close:
        if self.verbose:
            print(f"\n 🇩🇪 MAX HOLD: Closing {len(force_close)} aged position")
        for p in force_close:
            self._exit_position(p, 'max_hold_close')

    self.positions = [p for p in self.positions if not p.get('_force_close')]
    self.state.position_count = len(self.positions)

    self.days_active += 1
    self.state.market_day += 1

    # Starvation check (3 trades minimum)
    if self.state.entries_today < 3:
        self.state.days_without_trading += 1
    else:
        self.state.days_without_trading = 0

    # Injury healing
    heal_rate = 0.2 if self.state.trades_today == 0 else 0.08
    self.state.injury_level = max(0, self.state.injury_level - heal_rate)

    # SLEEP CONSOLIDATION - this is where constraints form
    episodes_before = len(self.memory.episodes)
    constraints_before = len(self.memory.constraints)
    skills_before = len(self.memory.skills)

```

```

self.memory consolidate()

new_constraints = len(self.memory.constraints) - constraints_before
new_skills = len(self.memory.skills) - skills_before

# Reset daily counters
self.state.trades_today = 0
self.state.entries_today = 0
self.state.todays_pnl = 0
self.constraint_violations_today = 0

# Energy recovery (sleep)
self.state.hunting_energy = min(1.0, self.state.hunting_energy + 0.6)

portfolio_dollars = self.state.portfolio_value * self.state.starting_capi

if self.verbose:
    print(f"\n🌙 DAY {self.days_active} EOD")
    print(f" Portfolio:    ${portfolio_dollars:,.2f} ({self.state.port
    print(f" Drawdown:      {self.state.current_drawdown:.1%}")
    print(f" Win Rate:       {self.state.win_rate:.1%} ({self.state.winni
    print(f" Efficiency:     {self._compute_efficiency():.2f}")
    print(f" Regime:        {self.current_macro.get('regime', '?')} | VI
    print(f" Constraints:    {len(self.memory.constraints)} (+{new_constr
    print(f" Skills:         {len(self.memory.skills)} (+{new_skills} tod
    print(f" Episodes:     {len(self.memory.episodes)}")

# -----
# EFFICIENCY & STATUS
# -----

def _compute_efficiency(self) -> float:
    """
    Profit efficiency: (gross profit - gross loss) / total risk taken.
    A measure of how much return per unit of risk deployed.
    Higher = organism is hunting more selectively with better outcomes.
    """
    if self.total_risk_taken == 0:
        return 0.0

    net_return = (self.state.portfolio_value - 1.0) * self.state.starting_cap
    return net_return / max(0.01, self.total_risk_taken * self.state.starting

def get_status(self) -> Dict:
    portfolio_dollars = self.state.portfolio_value * self.state.starting_capi
    return {
        'alive': self.state.is_viable(),

```



```

        'mode': self.mode.value,
        'portfolio_value': self.state.portfolio_value,
        'portfolio_dollars': portfolio_dollars,
        'pnl_dollars': portfolio_dollars - self.state.starting_capital,
        'pnl_pct': (self.state.portfolio_value - 1.0),
        'drawdown': self.state.current_drawdown,
        'win_rate': self.state.win_rate,
        'total_trades': self.state.total_trades,
        'efficiency': self._compute_efficiency(),
        'constraints_learned': len(self.memory.constraints),
        'skills_developed': len(self.memory.skills),
        'total_episodes': len(self.memory.episodes),
        'days_active': self.days_active,
        'regime': self.current_macro.get('regime', 'unknown'),
        'injury_level': self.state.injury_level,
        'confidence': self.state.confidence,
    }

def print_status(self):
    s = self.get_status()
    print(f"\n{'='*55}")
    print(f"  ORCA V2 STATUS - Day {self.days_active}")
    print(f"{'='*55}")
    print(f"  💰 Portfolio:      ${s['portfolio_dollars']:.2f} ({s['pnl_pct']}:
    print(f"  📉 Max Drawdown: {s['state.max_drawdown']:.1%}")
    print(f"  🎯 Win Rate:      {s['win_rate']:.1%} ({s['state.winning_trade
    print(f"  ⚡ Efficiency:    {s['efficiency']:.3f}")
    print(f"  🧠 Constraints:   {s['constraints_learned']}")
    print(f"  🏆 Skills:        {s['skills_developed']}")
    print(f"  📖 Episodes:      {s['total_episodes']}")
    print(f"  🌐 Regime:        {s['regime']}")
    print(f"  💪 Confidence:    {s['confidence']:.2f}")
    print(f"  🤕 Injury:        {s['injury_level']:.2f}")
    print(f"{'='*55}\n")

```